LVGL 开发实例



正点原子公司名称 ：广州市星翼电子科技有限公司

原子哥在线教学平台：www.yuanzige.com

开源电子网 / 论坛 ：http://www.openedv.com/forum.php

正点原子淘宝店铺 ：https://openedv.taobao.com

正点原子官方网站 ：www.alientek.com

正点原子 B 站视频 ：https://space.bilibili.com/394620890

电话：020-38271790 传真：020-36773971

请关注正点原子公众号，资料发布更新我们会通知。



扫码关注正点原子公众号

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	初稿：	dengzhimao	正点原子 linux 团队	2025.04.21

目录

前言 5
第一章 LVGL 简介 7
1.1 LVGL 简介 8
1.2 LVGL 库配置 8
1.3 SDK LVGL 开发参考 8
1.4 LVGL 在线文档教程 8
1.5 正点原子 LVLGL 视频教程 9
1.6 LVGL 如何启动 9
1.7 SDK 如何设置默认启动 LVGL 10
第二章 第一个 LVGL 项目 12
2.1 安装交叉编译器 13
2.2 第一个 LVGL 项目 13
2.3 交叉编译 LVGL 项目 14
第三章 基于 LVGL 的 LED 点亮实现 16
3.1 项目介绍 17
3.2 代码赏析 17

前言

本文档介绍 LVGL 的配置和启动 LVGL DEMO 介绍，指导用户在开发 LVGL 参考在线文档。提供开发示例，方便用户参考。

免责声明

本文档所提及的产品规格和使用说明仅供参考，如有内容更新，恕不另行通知；除非有特殊约定，本文档仅作为产品指导，所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有，未经公司的书面许可，任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息，请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

第一章 LVGL 简介

本章介绍 LVGL SDK 中的配置，讲解 LVGL 启动脚本，指导用户参考在线文档。

1.1 LVGL 简介

LVGL 是一个较为流行的免费和开源的嵌入式图形库，可以在 MCU、MPU 以及各种屏幕上绘制美观的 UI。

LVGL 是完全开源的，除了部分扩展功能外没有外部依赖，这使得它的移植非常简单。它可与任何现代 MCU 或 MPU 配合使用，并可与任何 (RT)OS 或裸机设置一起使用，以驱动电子纸、单色、OLED 或 TFT 显示器，甚至监视器。因此，甚至可以在 Windows、Linux 上调试 UI 界面，有一些基础的平台无关的 UI 调试，反复编译烧录固件是十分繁琐的，这时就可以在 PC 上搭建环境调试，再拷贝到嵌入式设备上编译运行。

——上面引用 RK 文档的简介。

总结，LVGL 是轻量级的跨平台 UI 库，想要在开发板上运行需要按第二章 LVGL 项目模板，安装交叉编译器后，交叉编译项目再拷贝到开发板上运行 LVGL。

1.2 LVGL 库配置

SDK/buildroot/configs/atk_dlrk3506_defconfig 中已经默认配置了，默认使用的 LVGL 版本为 v8.4，并且使用的是 LVGL 官方库。编译之后生成

/usr/lib/liblv_drivers.so

/usr/lib/liblvgl.so

两个库。我们基于项目直接链接到这两个库即可，我们会提供例子，用户直接基于例子在 main 函数里开发 LVGL 即可，非常方便！详细请看下面第二章中的例子。

```
#include "base/base.config"
#include "chips/rk3506_arm.config"
#include "fs/vfat.config"
#include "wifibt/bt.config"
#include "wifibt/wireless.config"
#include "multimedia/audio.config"
#include "wifibt/bt.config"
#include "wifibt/wireless.config"
#include "multimedia/gst/camera.config"
#include "lvgl.config"
# BR2_PACKAGE_CHRONY is reset to default
# BR2_PACKAGE_DNSMASQ is reset to default
# BR2_PACKAGE_DROPBEAR is reset to default
BR2_PACKAGE_EVTEST=y
BR2_PACKAGE_HOST_LINUX_HEADERS_CUSTOM_6_1=y
BR2_PACKAGE_LIBCAP=y
BR2_PACKAGE_MHZ=y
BR2_PACKAGE_MTD=y
# BR2_PACKAGE_PULSEAUDIO is reset to default
BR2_PACKAGE_RIPGREP=y
BR2_PACKAGE_RKADK_EXAMPLES=y
BR2_PACKAGE_ROCKCHIP_RGA=y
```

1.3 SDK LVGL 开发参考

参考<SDK>/docs/cn/Linux/Graphics/Rockchip_Developer_Guide_Linux_LVGL_CN.pdf，强烈建议去看这个文档。

1.4 LVGL 在线文档教程

推荐！https://docs.lvgl.io/8.4 与开发板 LVGL 版本配套。LVGL 非常完善的教程，原汁原味，可以看到控件样式、代码等。直接看官方文档，加上在开发板上实践学习即可！如果用户看不懂英文，请安装网页翻译插件即可。

1.5 正点原子 LVLGL 视频教程

视频地址：https://www.bilibili.com/video/BV1CG4y157Px?t=2.3 正点原子手把手教你学 LVGL 图形界面编程，可参考其中的 1~2 讲，15~64 讲。视频为 STM32 LVGL 教程，但是在讲部件和编程思想这部分内容与 Linux 下的 LVGL 是一样的，可以作为参考。

1.6 LVGL 如何启动

由于出厂系统默认配置了 Qt，所以 LVGL 显示与 weston 显示冲突了。用户会看到 LVGL 界面显示了，1~2 后又黑屏了。

LVGL 的启动脚本为/etc/init.d/pre_init/S00-lv_demo。这个脚本中默认已经注释不让 LVGL 启动。

编辑/etc/init.d/pre_init/S10-lv_demo 脚本，开启 LVGL DEMO。

```
#! /bin/sh

start() {
        #lv_demo &
}

case "$1" in
  start)
        start
        ;;
  *)
        echo "Usage: $0 {start}"
        exit 1
esac

exit $?
~
~
```

将上图的“#”去除，保存即可。

```
#! /bin/sh

start() {
        lv_demo &
}

case "$1" in
  start)
        start
        ;;
  *)
        echo "Usage: $0 {start}"
        exit 1
esac

exit $?
```

我们需还要关闭 Qt 和 weston 启动，将下面两行移动到其他路径，如/目录不让它们启动即可。

```
mv /etc/init.d/S49weston /
mv /etc/init.d/S50systemui /
sync
```

```
root@ATK-DLRK3506:/# mv /etc/init.d/S49weston /
root@ATK-DLRK3506:/# mv /etc/init.d/S50systemui /
root@ATK-DLRK3506:/#
```

最后我们重启开发板就会看到一个 LVGL 官方音乐播放器界面，是 LVGL 官方写的，模拟的，没有实际功能。

1.7 SDK 如何设置默认启动 LVGL

在 SDK/buildroot/board/alientek/atk-dlrk3506/fs-overlay/etc/init.d/pre_init/S10lv_demo 文件中，将“#”注释打开，如下图。

```
#! /bin/sh

start() {
        echo "Starting lv_demo"
        lv_demo &
}

case "$1" in
  start)
        start
        ;;
  *)
        echo "Usage: $0 {start}"
        exit 1
esac

exit $?
```

还需要禁用 weston，用户将 SDK/buildroot/output/atk_dlrk3506/target/etc/init.d/S49weston（前提先编译过一次 SDK）拷贝到 SDK/buildroot/board/alientek/atk-dlrk3506/fs-overlay/etc/init.d/下。然后再修改 S49weston 启动脚本。添加“#”注释，不让 weston 启动。

```
#!/bin/sh
### BEGIN INIT INFO
# Provides:          weston
# Required-Start:    mountvirtfs
# Required-Stop:
# Should-Start:
# Should-Stop:
# Default-Start:     2 3 4 5
# Default-Stop:      0 1 6
# Short-Description: Linux weston daemon
### END INIT INFO

PATH="/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin"

# Load default env variables from profiles(e.g. /etc/profile.d/weston.sh)
. /etc/profile

start_weston()
{
        /usr/bin/weston 2>&1 | tee /var/log/weston.log &
}

stop_weston()
{
        killall weston
}

case "$1" in
        start)
                echo -n "starting weston... "
                #start_weston
                echo "done."
                ;;
        stop)
                echo -n "stoping weston... "
                stop_weston || true
                echo "done."
                ;;
        restart|reload)
                echo -n "stoping weston... "
                stop_weston

                while pgrep -x weston; do
                        sleep .1
                done
                echo "done."

                echo -n "starting weston... "
                start_weston
                echo "done."
                ;;
        *)
                echo "Usage: $0 {start|stop|restart}"
                exit 1
esac

exit 0
```

禁止 Qt Demo 启动，将 SDK/buildroot/board/alientek/atk-dlrk3506/fs-overlay/etc/init.d/的 S50systemui 修改如下图。也是添加一个“#”注释启动的脚本即可。

```
# Provides:          systemui
# Required-Start:    mountvirtfs
# Required-Stop:
# Should-Start:
# Should-Stop:
# Default-Start:     2 3 4 5
# Default-Stop:      0 1 6
# Short-Description: Qt systemui
### END INIT INFO

PATH="/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin"

# Load default env variables from profiles(e.g. /etc/profile.d/*.sh)
. /etc/profile

start_systemui()
{
        # Wait for weston ready, 5 seconds timeout
        counter=0
        while [ ! -e "${XDG_RUNTIME_DIR}/wayland-0" ] && [ $counter -lt 50 ]; do
                sleep 0.1
                counter=$((counter + 1))
        done
        /userdata/ui/systemui > /dev/null 2>&1 &
}

stop_systemui()
{
        killall systemui
}

case "$1" in
        start)
                echo -n "starting systemui... "
                #start_systemui
                echo "done."
                ;;
        stop)
                echo -n "stoping systemui... "
                stop_systemui
                echo "done."
                ;;
        restart|reload)
                echo -n "stoping systemui... "
                stop_systemui && sleep .3
                echo "done."

                echo -n "starting systemui... "
                start_systemui
                echo "done."
                ;;
        *)
                echo "Usage: $0 {start|stop|restart}"
                exit 1
esac

exit 0
```

最后在 SDK 顶层目录执行./build.sh rootfs 编译 rootfs 即可。

第二章 第一个 LVGL 项目

本章提供一个 LVGL 项目模板，用户直接基于这个模板编写 LVGL 程序。然后交叉编译到开发板上运行。

2.1 安装交叉编译器

参考开发板网盘 A 盘 10、用户手册/辅助文档/ 【正点原子】基于 Buildroot 系统_交叉编译器安装与使用参考手册。

注意：必须先安装交叉编译器，交叉编译后的项目才可以拷贝到开发板上运行。

2.2 第一个 LVGL 项目

例程位于开发板网盘 A 盘-基础资料/程序源码/lvgl 开发实例/lvgl_demo.tar.gz/01_lvgl_button。拷贝到 Ubuntu 下，使用 vscode 来编辑代码（安装 VSCODE 请查看开发板网盘 A 盘-基础资料/用户手册/辅助文档/【正点原子】Ubuntu 环境搭建&VSCode 安装使用）。

如下图，笔者已经拷贝到 Ubuntu 下，并且使用 tar 指令解压项目得到如下文件。

```
tar xf lvgl_demo.tar.gz
```

```
alientek@alientek:~/lvgl$ ls
01_lvgl_button  02_lvgl_button_led  lvgl_demo.tar.gz
alientek@alientek:~/lvgl$
```

01_lvgl_button 项目文件介绍：

```
alientek@alientek:~/lvgl/01_lvgl_button$ tree -L 3
.
├── CMakeLists.txt
├── common
│   ├── kalman_filter.c
│   ├── kalman_filter.h
│   ├── ui_scaler.c
│   └── ui_scaler.h
├── lvgl8
│   ├── evdev.c
│   ├── evdev.h
│   ├── key.c
│   ├── key.h
│   ├── lv_port_disp.c
│   ├── lv_port_disp.h
│   ├── lv_port_indev.c
│   ├── lv_port_indev.h
│   ├── lv_port_init.c
│   └── lv_port_init.h
├── main.c
├── main.h
└── sys
    ├── cpu.c
    ├── cpu.h
    ├── soc.c
    ├── soc.h
    ├── timestamp.c
    ├── timestamp.h
    ├── trace.c
    └── trace.h

3 directories, 25 files
alientek@alientek:~/lvgl/01_lvgl_button$
```

用于管理项目，生成Makefile编译

一些公共文件

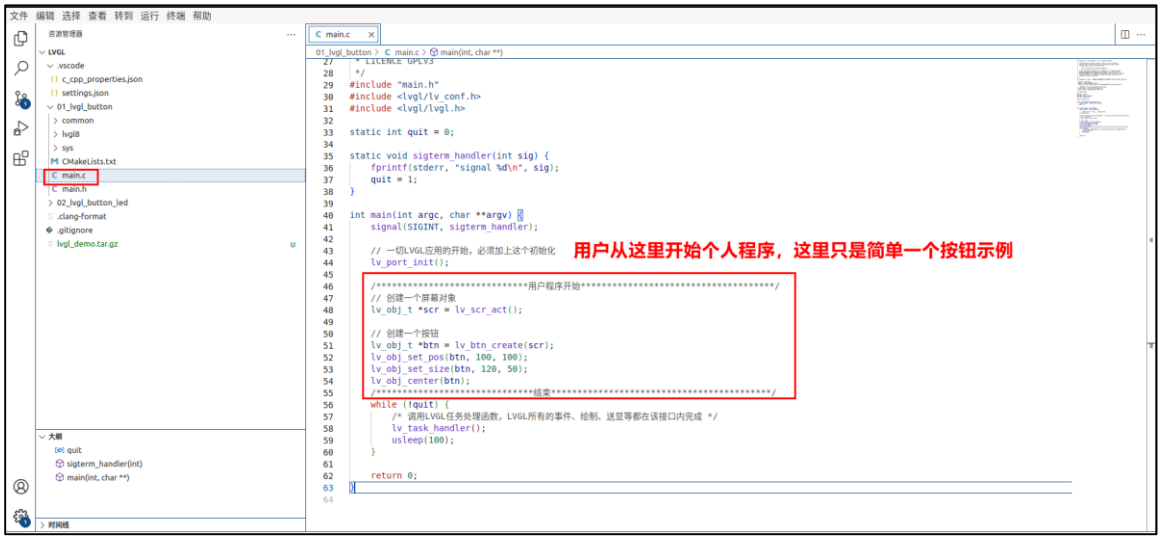
LVGL初始化的一些文件，比较重要，如初如化显示设备

用户主程序，用户主要从main.c入手

系统文件，仅用到里面的函数

所以用户需要以 01_lvgl_button 为模板。拷贝 01_lvgl_button 为 test。然后在 main.c 中开发用户个人的 LVGL 程序即可，还需要修改的是 CMakeLists.txt 中的 project(01_lvgl_button)项目名字即可。common，lvgl8，sys，这些一般不用修改，这些适配 RK3506 的公共代码，**每个项目必须包含它们**。并且在 RK3506 中，在第 1.2 小节已经提及过了，LVGL 官方代码已经编译成 so 库了，这样在构建 LVGL 项目时不用重复编译了，我们链接到 so 库会非常简单。

如下图，笔者用 vscode 打开项目文件夹，打开 main.c，如下图所示，用户主要在 main.c 中标记位置开始个人程序即可，项目比较简单，显示一个按钮而已。



2.3 交叉编译 LVGL 项目

```
cd 01_lvgl_button  # 进行第一个项目路径下
cmake .            # 当前路径必须有 CMakeLists.txt
make -j 16         # 开始编译
file 01_lvgl_button # 查看编译生成的文件架构为 ARM 说明为板子上运行的程序
make distclean     # 用于清除编译生成的文件（可选）
```

```
alientek@alientek:~/lvgl/01_lvgl_button$ cmake .
-- The C compiler identification is GNU 9.4.0
-- The CXX compiler identification is GNU 9.4.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/alientek/lvgl/01_lvgl_button
alientek@alientek:~/lvgl/01_lvgl_button$ make -j 16
Scanning dependencies of target 01_lvgl_button
[ 15%] Building C object CMakeFiles/01_lvgl_button.dir/lvgl8/evdev.c.o
[ 23%] Building C object CMakeFiles/01_lvgl_button.dir/lvgl8/lv_port_indev.c.o
[ 23%] Building C object CMakeFiles/01_lvgl_button.dir/lvgl8/key.c.o
[ 30%] Building C object CMakeFiles/01_lvgl_button.dir/lvgl8/lv_port_disp.c.o
[ 46%] Building C object CMakeFiles/01_lvgl_button.dir/lvgl8/lv_port_init.c.o
[ 46%] Building C object CMakeFiles/01_lvgl_button.dir/sys/timestamp.c.o
[ 53%] Building C object CMakeFiles/01_lvgl_button.dir/common/kalman_filter.c.o
[ 61%] Building C object CMakeFiles/01_lvgl_button.dir/sys/cpu.c.o
[ 69%] Building C object CMakeFiles/01_lvgl_button.dir/sys/trace.c.o
[ 76%] Building C object CMakeFiles/01_lvgl_button.dir/sys/soc.c.o
[ 84%] Building C object CMakeFiles/01_lvgl_button.dir/common/ui_scaler.c.o
[ 92%] Building C object CMakeFiles/01_lvgl_button.dir/main.c.o
[100%] Linking C executable 01_lvgl_button
[100%] Built target 01_lvgl_button
alientek@alientek:~/lvgl/01_lvgl_button$ file 01_lvgl_button
01_lvgl_button: ELF 32-bit LSB shared object, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-armhf.so.3, for GNU/Linux 3.2.0, not stripped
alientek@alientek:~/lvgl/01_lvgl_button$
```

上面生成的可执行文件 01_lvgl_button 拷贝到开发板路径下执行即可。拷贝方法参考开发板网盘 A 盘-基础资料/用户手册/辅助文档/【正点原子】Ubuntu&Windows&Linux 开发板互传文件参考手册。或者【正点原子】NFS 搭建手册，将 Ubuntu 目录挂载到开发板，这样就不用手动拷贝了。再或者【正点原子】adb 工具使用说明使用 adb 上传到开发板。方法多种多样，关键用户需要熟练掌握。辅助文档基本都是笔者编写的，团队审核，所以非常熟悉。

```
alientek@alientek:~/lvgl/01_lvgl_button$ ls
01_lvgl_button  CMakeCache.txt  CMakeFiles  cmake_install.cmake  CMakeLists.txt  common  lvgl8  main.c  main.h  Makefile  sys
alientek@alientek:~/lvgl/01_lvgl_button$
```

如下图，笔者将 lvgl 文件夹挂载到开发板路径下，所以直接执行./01_lvgl_button 即可运行程序。

```
root@ATK-DLRK3506:/lvgl/01_lvgl_button# ./01_lvgl_button
LV_DRIVERS_SET_PLANE not be set, use DRM_PLANE_TYPE_PRIMARY
Best plane: 57
disp_init res1:[720 x 1280@60fps] bit depth 32
DRM subsystem and buffer mapped successfully lcd_fps:60
evdev_get_tp_event: /sys/class/input/input0/name = dsi1_ts_gt9xx
/dev/input/event0
EV_ABS ABS_MT_POSITION_X
        Min          0
        Max        720
EV_ABS ABS_MT_POSITION_Y
        Min          0
        Max       1280
evdev_calibrate = 0
```

第三章 基于 LVGL 的 LED 点亮实现

在嵌入式系统开发中，图形用户界面（GUI）与硬件交互的融合已成为提升产品智能化水平的关键。LVGL（Light and Versatile Graphics Library）作为专为资源受限设备设计的开源图形库，凭借其轻量化架构与跨平台特性，在工业控制、智能家居、可穿戴设备等领域得到广泛应用。本项目以 LED 控制为切入点，通过 LVGL 实现图形界面与硬件驱动的协同，验证了嵌入式 GUI 在低功耗设备上的实用价值。

3.1 项目介绍

项目位于开发板网盘 A 盘-基础资料/程序源码/lvgl 开发实例/lvgl_demo.tar.gz/02_lvgl_button_led。

这是一个模拟智能家居控制面板的界面，按钮与屏幕同大小，点击即可控制开发板上的 LED。

3.2 代码赏析

```
1   /*
2    * Copyright (c) 2021 Rockchip, Inc. All Rights Reserved.
3    *
4    *  Licensed under the Apache License, Version 2.0 (the "License");
5    *  you may not use this file except in compliance with the License.
6    *  You may obtain a copy of the License at
7    *
8    *     http://www.apache.org/licenses/LICENSE-2.0
9    *
10   *  Unless required by applicable law or agreed to in writing,
software
11   *  distributed under the License is distributed on an "AS IS"
BASIS,
12   *  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or
implied.
13   *  See the License for the specific language governing permissions
and
14   *  limitations under the License.
15   */
16  /*
17   * Copyright (c) 2025 广州市星翼电子科技有限公司（正点原子）All Rights
Reserved.
18   *
19   * @author: Deng Zhimao
20   * @email: dengzhimao@alientek.com
21   * B站作品:
https://space.bilibili.com/474103963?spm_id_from=333.788.0.0
22
23   * 开源电子网: http://www.openedv.com/forum.php
24   * 正点原子官网: https://www.alientek.com
25   * 店铺: https://zhengdianyuanzi.tmall.com
26   *
27   * LICENCE GPLV3
28   */
29
```

```
30  #include "main.h"
31  #include <lvgl/lv_conf.h>
32  #include <lvgl/lvgl.h>
33  static int quit = 0;
34
35  // 字库结构体实例
36  lv_ft_info_t ft_info;
37  // 按钮
38  static lv_obj_t *btn;
39  // 标签
40  static lv_obj_t *label;
41
42  // 开发板中字体库路径
43  #define FREETYPE_FONT_FILE ("/usr/share/fonts/AlibabaPuHuiTi-2-105-
Heavy.otf")
44
45  #define LED_BRIGHTNESS_PATH "/sys/class/leds/work/brightness"
46
47  static void sigterm_handler(int sig) {
48      fprintf(stderr, "signal %d\n", sig);
49      quit = 1;
50  }
51
52  // 读取 led 状态
53  int read_brightness() {
54      FILE *file = fopen(LED_BRIGHTNESS_PATH, "r");
55      if (!file) {
56          LV_LOG_USER("Failed to open brightness file for reading");
57          return -1;
58      }
59      int brightness;
60      if (fscanf(file, "%d", &brightness) != 1) {
61          LV_LOG_USER("Failed to read brightness value");
62          fclose(file);
63          return -1;
64      }
65      fclose(file);
66      return brightness;
67  }
68
69  // 控制 led
70  int write_brightness(int brightness) {
71      FILE *file = fopen(LED_BRIGHTNESS_PATH, "w");
```

```
72      if (!file) {
73          LV_LOG_USER("Failed to open brightness file for writing");
74          return -1;
75      }
76
77      if (fprintf(file, "%d\n", brightness) < 0) {
78          LV_LOG_USER("Failed to write brightness value");
79          fclose(file);
80          return -1;
81      }
82
83      fclose(file);
84      return 0;
85  }
86
87  static void event_handler(lv_event_t *e) {
88      lv_event_code_t code = lv_event_get_code(e);
89      // 获取触发事件的对象
90      lv_obj_t *target = lv_event_get_target(e);
91
92      if (code == LV_EVENT_CLICKED) {
93          LV_LOG_USER("Clicked");
94          if (target == btn) {
95              LV_LOG_USER("btn Clicked");
96          }
97      }
98
99      if (code == LV_EVENT_VALUE_CHANGED) {
100         // 根据按钮状态更新标签文本
101         if (lv_obj_has_state(target, LV_STATE_CHECKED)) {
102             lv_label_set_text(label, "关灯");
103             write_brightness(1);
104         } else {
105             lv_label_set_text(label, "开灯");
106             write_brightness(0);
107         }
108     }
109 }
110
111 int main(int argc, char **argv) {
112     signal(SIGINT, sigterm_handler);
113
114     // 一切 LVGL 应用的开始，必须加上这个初始化
```

```
115     lv_port_init();
116
117     /*****用户程序开始*****/
118     ft_info.name = FREETYPE_FONT_FILE; // 使用绝对路径指定字体文件
119     ft_info.weight = 50;               // 字体大小
120     ft_info.style = FT_FONT_STYLE_NORMAL;
121     ft_info.mem = NULL;
122
123     // 初始化字体
124     if (!lv_ft_font_init(&ft_info)) {
125         printf("create failed.");
126     }
127
128     // 将心跳改为手动触发
129     system("echo none > /sys/class/leds/work/trigger");
130
131     int brightness = read_brightness();
132     if (brightness != -1) {
133         LV_LOG_USER("Current brightness: %d\n", brightness);
134     }
135
136     // 创建一个样式用于设置默认状态
137     static lv_style_t style_default;
138     lv_style_init(&style_default);
139     lv_style_set_bg_opa(&style_default, LV_OPA_COVER);
140     lv_style_set_bg_color(&style_default,
                          lv_palette_main(LV_PALETTE_GREY)); // 默认状态颜色
141     lv_style_set_radius(&style_default, 0);
142
143     // 创建一个样式用于设置按下状态
144     static lv_style_t style_pressed;
145     lv_style_init(&style_pressed);
146     lv_style_set_bg_opa(&style_pressed, LV_OPA_COVER);
147     lv_style_set_bg_color(&style_pressed,
                          lv_palette_main(LV_PALETTE_BLUE)); // 按下状态颜色
148     lv_style_set_radius(&style_pressed, 0);
149
150     // 创建一个按钮
151     btn = lv_btn_create(lv_scr_act());
152     lv_obj_set_size(btn, lv_obj_get_width(lv_scr_act()),
                            lv_obj_get_height(lv_scr_act()));
153     // 将样式应用到按钮
154     lv_obj_add_style(btn, &style_default, LV_STATE_DEFAULT);
```

```
155     lv_obj_add_style(btn, &style_pressed, LV_STATE_CHECKED);
156     label = lv_label_create(btn);
157     lv_obj_set_style_text_font(label, ft_info.font,
                                   LV_STATE_DEFAULT);
158     lv_obj_add_flag(btn, LV_OBJ_FLAG_CHECKABLE);
159     lv_obj_center(label);
160 
161     if (brightness > 0) {
162         lv_label_set_text_fmt(label, "关灯");
163     } else {
164         lv_label_set_text_fmt(label, "开灯");
165     }
166     lv_obj_add_state(btn, brightness > 0 ? LV_STATE_CHECKED :
                                           LV_STATE_DEFAULT);
167 
168     lv_obj_add_event_cb(btn, event_handler, LV_EVENT_ALL, NULL);
169 
170     /********结束*********/
171     while (!quit) {
172         /* 调用 LVGL 任务处理函数，LVGL 所有的事件、绘制、送显等都在该接口内完成
*/
173         lv_task_handler();
174         usleep(100);
175     }
176 
177     return 0;
178 }
```

上面的代码非常简单，LVGL 与硬件交互代码主要在 52~109 行。上述代码有详细的注释，用户自行查看。

好了，现在结合 https://docs.lvgl.io/8.4 在线文档愉快的开始我们 LVGL 项目吧！